JAUME VILADOMS
Formació Professional integrada

Activitat 6

Modularitzar el codi

De tres fitxers solts a un projecte

Mòdul	M0613 · Desenvolupament web en entorn servidor
Cicle	CFGS Desenvolupament d'Aplicacions Web · 2n curs
Resultats d'aprenentatge	RA2 · RA3
Durada	3 hores
Curs	2026-2027

Presentació de l'activitat

Objectiu	RA2. Escriu sentències executables per un servidor web reconeixent i aplicant procediments d'integració del codi en llenguatge de marques. RA3. Escriu blocs de sentències embeguts en llenguatges de marques, seleccionant i utilitzant les estructures de programació.
Requisits	L'Activitat 3 acabada, Visual Studio Code o editor de text i el servidor local (WAMP/XAMPP) en marxa.
Entrega	Lliurar un fitxer ZIP a Clickedu amb tots els documents generats.

En aquesta activitat no escriuràs cap funcionalitat nova. Agafaràs el codi que ja vas entregar a l'Activitat 3 i el reorganitzaràs: el mateix formulari, la mateixa validació i el mateix resum, però repartits en fitxers que tenen sentit.

Això es diu modularitzar i és el que separa tres fitxers solts d'un projecte. Quan acabis, canviar el nom de la botiga serà tocar una línia en comptes de buscar-la per tots els fitxers, i fer una pàgina nova et costarà vuit línies.

> **Parteixes de la teva Activitat 3:** copia-la sencera a una carpeta nova i treballa-hi a sobre. No esborris ni modifiquis l'entrega original, perquè al final hauràs de comparar les dues versions.

Com quedarà el projecte

Al final tindràs vuit fitxers en lloc de tres. Quatre són d'infraestructura i tres són pàgines:

Fitxer	Què conté	Apartat
config.php	Els valors que es fan servir a més d'un lloc	2
funcions.php	Totes les funcions pròpies	3
header.php	El disseny de dalt de qualsevol pàgina	4
footer.php	El disseny de baix de qualsevol pàgina	4
registre.php	El formulari (abans era registre.html)	1 i 5
processa.php	Només la lògica: recollir, validar i mostrar	5
gracies.php	Una pàgina nova, feta de zero	6
estils.css	El full d'estils, sense canvis	—

> **Important:** els dos primers fitxers, config.php i funcions.php, no han d'escriure res a la pantalla. Ni un echo, ni un espai en blanc abans del <?php. Si ho fan, trencaran totes les pàgines que els incloguin.

Apartat 1 · Preparar la carpeta

Crea una carpeta nova anomenada M0613_ACT06 dins del directori del servidor i copia-hi els tres fitxers de l'Activitat 3.

Canviar l'extensió del formulari

El primer canvi és el més petit i el més important: registre.html ha de passar a dir-se registre.php.

Un fitxer .html el servidor l'envia tal com està, sense mirar-ne el contingut. Un .php el passa primer per l'intèrpret de PHP. Com que el formulari haurà d'incloure la capçalera i el peu, ha de ser .php per força.

> **Compte amb els enllaços:** a processa.php hi ha dos enllaços que apunten a registre.html. Si no els canvies, et donaran error 404. Busca'ls i actualitza'ls tots dos.

Comprovació abans de continuar

Obre http://localhost/M0613_ACT06/registre.php i envia el formulari. Ha de funcionar exactament igual que a l'Activitat 3. Si no, atura't aquí: la resta de l'activitat es construeix a sobre d'això.

Apartat 2 · config.php

Crea el fitxer config.php. Aquí hi va tot allò que apareix a més d'un lloc i que algun dia pot canviar.

La prova per saber si una cosa va a config.php és aquesta: si demà canviés, quantes vegades l'hauries de tocar? Si la resposta és més d'una, va a config.php.

config.php

```php
<?php
/**
 * Configuració general del projecte.
 * Aquest fitxer NO escriu res a la pantalla.
 */

// Dades de la botiga
const BOTIGA_NOM    = 'BotigaMolona';
const BOTIGA_LEMA   = 'La teva botiga molona;
const BOTIGA_CORREU = 'info@botigamolona.cat';

// Regles de validació
const MIN_USUARI = 4;
const MIN_CLAU   = 6;

// Rutes
const FULL_ESTILS   = 'estils.css';
const PAG_FORMULARI = 'registre.php';
```

Fer-lo servir

Un cop tinguis les constants, busca pel teu codi tots els llocs on hi ha el text escrit a mà i canvia'ls per la constant. Com a mínim:

- El nom de la botiga, que surt al <title>, a l'<h1> de la capçalera i al peu de pàgina.
- El lema de la botiga, sota el títol.
- Les mides mínimes de la validació: on ara hi ha strlen($clau) < 6 hi ha d'anar strlen($clau) < MIN_CLAU.

> **Per què val la pena:** amb MIN_CLAU definida un sol cop, el missatge d'error i l'etiqueta del formulari sempre diran el mateix número. Si ho tens escrit a mà en dos llocs, tard o d'hora un dirà 6 i l'altre 8.

Constants, no variables

Fixa't que es fan servir constants (const) i no variables. Una constant no es pot canviar un cop definida, i això és precisament el que volem: ningú no ha de poder canviar el nom de la botiga a mitja pàgina.

```
echo BOTIGA_NOM;     // sense dolar i sense cometes
echo $botiga;        // això seria una variable, no es el cas
```

Apartat 3 · funcions.php

Crea el fitxer funcions.php. Serà el lloc on viuran totes les funcions pròpies del projecte, i de moment n'hi escriuràs dues de noves.

Totes dues surten del mateix lloc: coses que a l'Activitat 3 vas escriure repetides o a mig fer, i que ara poden tenir un sol lloc. Són les funcions que vas aprendre a escriure a l'Activitat 5, aplicades al projecte de debò.

A processa.php, la línia que les porta:

```
require_once 'funcions.php';
```

> **Totes dues han de portar:** tipus als paràmetres, tipus de retorn i un bloc de comentari PHPDoc. Si no saps què és això últim, llegeix l'apartat següent abans de continuar.

Què és el PHPDoc

El PHPDoc és un bloc de comentari que es posa just a sobre d'una funció i que explica tres coses: què fa, què rep i què torna. S'escriu amb una forma concreta perquè no és només per a les persones: els editors de codi el saben llegir.

Es reconeix perquè obre amb dues asterisques i perquè cada línia de dins comença amb una asterisca:

```
/**                  <- obre amb DUES asteriscos
 * Que fa la funció.     <- descripció, en una línia
 *
 * @param tipus $nom  Per a que serveix el paràmetre
 * @return tipus      Que torna la funció
 */
```

Les parts que comencen amb @ es diuen etiquetes i són les que donen la informació concreta:

Etiqueta	Quantes n'hi ha	Què hi poses
(descripció)	Una, la primera línia	Què fa la funció, en una frase i començant amb un verb
@param	Una per cada paràmetre, en ordre	El tipus, el nom del paràmetre i per a què serveix
@return	Una, si la funció torna alguna cosa	El tipus i què representa el valor que torna

Un exemple sencer

Aquesta és una funció petita amb el seu PHPDoc complet. Fixa-t'hi en com cada @param correspon a un paràmetre de la línia de sota:

exemple.php

```php
/**
 * Calcula el preu final d'un producte amb l'IVA inclòs.
 *
 * @param float $preu Preu del producte sense IVA
 * @param float $iva  Percentatge d'IVA a aplicar (21 vol dir 21%)
 * @return float      Preu amb IVA, arrodonit a dos decimals
 */
function preuAmbIva(float $preu, float $iva): float
{
    return round($preu * (1 + $iva / 100), 2);
}
```

Per a què serveix de debò

Podries pensar que és feina de més per a res, perquè PHP ignora els comentaris i el programa funciona igual. El guany és un altre:

- L'editor te'l mostra. Si poses el ratolí a sobre d'una crida a preuAmbIva(), Visual Studio Code t'ensenya la descripció i els paràmetres sense que hagis d'anar a buscar la funció.
- Mentre escrius la crida, l'editor et va dient quin paràmetre toca i què hi va.
- Al cap de tres mesos, el que llegirà aquella funció seràs tu mateix, i no te'n recordaràs.
- És l'estàndard de PHP. Qualsevol codi professional que obris el porta, i les llibreries que facis servir també.

Truc: si tens instal·lada una extensió de PHP a Visual Studio Code, sovint n'hi ha prou d'escriure /** just a sobre de la funció i prémer Enter perquè et generi l'esquelet amb els @param ja posats. Tu només has d'omplir les descripcions.

El tipus del @param ha de ser el de debò. Si la funció rep un string i al PHPDoc hi poses int, has escrit una mentida que l'editor es creurà i que et confondrà d'aquí a un mes. Un PHPDoc equivocat és pitjor que no tenir-ne.

Funció 1 · camp()

A l'Activitat 3 vas escriure dotze vegades aquesta mateixa línia, canviant-ne només el nom del camp:

```
$nom     = trim($_POST['nom'] ?? '');
$cognoms = trim($_POST['cognoms'] ?? '');
$correu  = trim($_POST['correu'] ?? '');
// ... i així nou vegades més
```

Dotze còpies de la mateixa idea. Si un dia decideixes que, a més del trim(), també vols treure els salts de línia, has de tocar dotze línies i no deixar-te'n cap. Converteix-la en una funció:

```
/**
 * Recull un camp de text del formulari ja net d'espais.
 *
 * @param string $camp    Nom del camp (l'atribut name del formulari)
 * @param string $defecte Valor si el camp no ha arribat
 * @return string         El valor net
 */
function camp(string $camp, string $defecte = ''): string
{
    return trim($_POST[$camp] ?? $defecte);
}
```

A partir d'aquí, l'array de dades queda molt més curt de llegir:

```
$usuari = [
    'nom'     => camp('nom'),
    'cognoms' => camp('cognoms'),
    'origen'  => camp('origen', 'no ho ha dit'),
    // ... la resta de camps
];
```

Les caselles de verificació no: una casella que no es marca no s'envia, i per tant camp() no serveix per recollir-la. Aquestes es continuen recollint amb isset(), com a l'Activitat 3.

Funció 2 · etiqueta()

A la taula del resum, les capçaleres surten amb el nom intern del camp: cp, naixement, adreça. Això ho entens tu, que has escrit l'array, però no l'usuari que llegeix la pàgina.

Escriu una funció que rebi la clau i torni un text llegible:

```
/**
 * Converteix la clau de l'array en una etiqueta llegible.
 *
 * @param string $clau Clau de l'array ('cp', 'naixement'...)
 * @return string      Etiqueta per mostrar a la taula
 */
function etiqueta(string $clau): string
{
    $noms = [
        'nom'       => 'Nom',
        'naixement' => 'Data de naixement',
        'cp'        => 'Codi postal',
        // ... completa la resta de camps
    ];

    return $noms[$clau] ?? ucfirst($clau);
}
```

I a la taula del resum, canvia la capçalera de cada fila perquè la faci servir:

```
<?php foreach ($usuari as $clauCamp => $valor): ?>
  <tr>
    <th><?= etiqueta($clauCamp) ?></th>
    <td><?= htmlspecialchars($valor) ?></td>
  </tr>
<?php endforeach; ?>
```

Fixa-t'hi en el return: el ?? fa que, si la clau no és a la llista, la funció torni el nom del camp amb la primera lletra en majúscula en lloc de fallar. Una funció que no es trenca amb una entrada que no esperava val molt més que una que sí.

Per què require_once i no require

Aquesta és la part que has d'entendre, no només copiar. Si un fitxer amb funcions s'inclou dues vegades amb require, PHP intenta declarar les mateixes funcions dos cops i s'atura:

```
Fatal error: Cannot redeclare etiqueta() (previously declared in funcions.php:3)
```

Amb require_once, PHP es recorda dels fitxers que ja ha inclòs i la segona vegada no fa res. Per això la configuració i les funcions sempre van amb require_once.

Prova-ho tu: posa a propòsit dues línies de require 'funcions.php'; seguides, recarrega la pàgina i mira l'error que surt. Després torna-ho a deixar bé amb require_once. Val la pena haver vist aquest missatge una vegada a la vida.

Apartat 4 · header.php i footer.php

Aquests dos fitxers són el cor de l'activitat. Contenen el disseny que ara mateix tens copiat a les dues pàgines.

header.php

Hi va tot el que hi ha des del <!DOCTYPE> fins a l'obertura del <main>, les dues pàgines inclouen el mateix:

header.php

```
<?php
// Si una pàgina s'oblida del títol, no volem un avís a la pantalla.
if (!isset($titolPagina)) {
  $titolPagina = 'Inici';
}
?>
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="utf-8">
  <title><?= htmlspecialchars($titolPagina) ?> · <?= BOTIGA_NOM ?></title>
  <link rel="stylesheet" href="<?= FULL_ESTILS ?>">
</head>
<body>

  <header>
    <h1><?= BOTIGA_NOM ?></h1>
    <p><?= BOTIGA_LEMA ?></p>
```

```
    </header>

    <main>
```

Fixa-t'hi: el fitxer obre etiquetes que no tanca. No és un error. El <main> i el <body> els tancarà footer.php. Entre els dos formen una pàgina sencera.

El títol de cada pàgina

header.php no rep cap paràmetre, però fa servir una variable que no té: $titolPagina. Funciona perquè el fitxer inclòs s'executa dins del que l'inclou i, per tant, en veu les variables.

Això vol dir que cada pàgina assigna el seu títol ABANS del require:

```
$titolPagina = 'Crea el teu compte';
require 'header.php';
```

Si ho fas a l'inrevés i assignes el títol després del require, PHP et dirà Undefined variable $titolPagina i a la pestanya del navegador hi sortirà Inici. És l'error més comú d'aquesta activitat.

footer.php

Tanca el que ha obert la capçalera i hi afegeix el peu de pàgina:

footer.php

```
    </main>

    <footer>
        <p><?= BOTIGA_NOM ?> &middot; Projecte de classe del M0613</p>
        <p>Contacte: <?= BOTIGA_CORREU ?></p>
    </footer>

</body>
</html>
```

Apartat 5 · Muntar les dues pàgines

Ara buida registre.php i processa.php de tot el disseny i deixa-hi només el que és propi de cada pàgina.

registre.php

registre.php

```
<?php
require_once 'config.php';

$titolPagina = 'Crea el teu compte';

require 'header.php';
?>

        <h2>Crea el teu compte</h2>
```

```
    <form action="processa.php" method="post">
      <!-- el formulari, igual que a l'Activitat 3 -->
    </form>

<?php
require 'footer.php';
```

processa.php i l'ordre de les coses

Aquí hi ha la part que s'ha de pensar. El títol de la pàgina depèn de si hi ha errors o no, i per tant s'ha de calcular abans d'incloure la capçalera. Si no, la capçalera ja haurà escrit el <title> i serà tard.

L'ordre del fitxer ha de ser exactament aquest:

1. Els require_once de config.php i funcions.php.
2. La comprovació del REQUEST_METHOD.
3. Recollir les dades i desar-les a l'array associatiu.
4. Validar-les i omplir l'array d'errors.
5. Calcular $titolPagina segons si hi ha errors o no.
6. Incloure la capçalera i, a partir d'aquí, escriure l'HTML.
7. Incloure el peu.

processa.php

```
<?php
require_once 'config.php';
require_once 'funcions.php';

// ... tota la lògica: recollir, validar ...

$titolPagina = count($errors) > 0
  ? 'No s\'ha creat el compte'
  : 'Compte creat';

require 'header.php';

?>

  <?php if (count($errors) > 0): ?>
    <!-- la llista d'errors -->
  <?php else: ?>
    <!-- el resum amb la taula -->
  <?php endif; ?>

<?php
require 'footer.php';
```

La pàgina d'accés directe

A l'Activitat 3, si algú escrivia l'adreça de processa.php directament, sortia un text pelat sense disseny. Ara que tens la capçalera i el peu, arreglar-ho és regalat:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {

    $titolPagina = 'Acces directe';
    require 'header.php';

    echo '<h2>Aqui no s\'hi arriba directament</h2>';
    echo '<a class="tornar" href="' . PAG_FORMULARI . '">Anar al formulari</a>';

    require 'footer.php';
    exit;
}
```

Apartat 6 · Una pàgina nova de zero

Aquest apartat és la prova que tota la feina ha servit d'alguna cosa. Fes una pàgina nova, gracies.php, a la qual s'arribi des del botó del resum quan el registre ha anat bé.

Ha de tenir exactament el mateix disseny que les altres dues: el mateix capçal, el mateix peu, el mateix full d'estils. I l'has de poder escriure sencera en menys de deu línies.

gracies.php

```
<?php
require_once 'config.php';

$titolPagina = 'Gracies';
require 'header.php';

?>

    <h2>Gràcies per registrar-te</h2>

    <div class="ok">
        <p>El teu compte de <?= BOTIGA_NOM ?> ja està actiu.
            Si tens cap problema, escriu-nos a <?= BOTIGA_CORREU ?>.</p>
    </div>

    <a class="tornar" href="<?= PAG_FORMULARI ?>">Registrar un altre compte</a>

<?php
require 'footer.php';
```

Després, al resum de processa.php, canvia l'enllaç final perquè porti aquí.

> **Atura't un moment i compara.** Obre la teva Activitat 3 i compta quantes línies hauries hagut d'escriure per fer aquesta mateixa pàgina. Aquesta diferència és tot el sentit de l'activitat d'avui.

Com s'entrega

Comprimeix la carpeta M0613_ACT06 sencera en un fitxer ZIP i puja'l a Clickedu. El ZIP ha de contenir els vuit fitxers i una carpeta de captures:

Fitxer	Contingut
config.php	Les constants, sense cap echo
funcions.php	Les quatre funcions amb el seu PHPDoc
header.php	El disseny de dalt, amb $titolPagina
footer.php	El disseny de baix
registre.php	El formulari muntat amb els includes
processa.php	Només la lògica, en l'ordre correcte
gracies.php	La pàgina nova
estils.css	El full d'estils
captures/	Tres captures de pantalla (vegeu més avall)

Les tres captures que has d'incloure:

- El formulari enviat amb errors, on es vegi que la pàgina d'error també té el disseny.
- Un registre correcte amb la taula del resum.
- La pàgina gracies.php.

> **Abans de comprimir:** passa per les tres pàgines amb el log d'errors obert. No hi ha d'haver cap warning ni cap notice, ni tan sols si entres a processa.php directament sense passar pel formulari.

Recorda que, a més, has de pujar els fitxers al teu repositori de Git, com fem amb totes les activitats:

```
git add .
git commit -m "M0613 ACT06: modularitzacio del projecte amb include i require"
git push
```

> **Es corregeix executant-ho.** Obriré les tres pàgines i, a més, canviaré el valor de BOTIGA_NOM a config.php. Si està ben fet, el nom ha de canviar a les tres pàgines alhora. Si he de tocar més d'un fitxer, l'apartat 2 no està superat.